Performance Testing

Date	03 July 2026
Team ID	[Enter Team ID]
Project Title	HDI Predictor
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Custom Python Benchmark Script (run_load_test.py)
Type of Testing	Load Testing, Concurrency Testing
Target Module	Flask Prediction API (/predict), User Input Form
Test Environment	Local System (Windows 11, Python 3.11, Flask)
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of virtual Users	Duration (sec) / Requests	Expected Outcome
1	Scenario 1: Baseline Request	1	10 requests	Prediction generated successfully, avg latency < 50 ms
2	Scenario 2: Load Testing	5	50 requests	Stable response time, no errors, throughput > 20 req/s
3	Scenario 3: Concurrency Spike	15	150 requests	Application remains responsive, error rate < 1%

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (pass/fail)	Remarks
1	Response Time (Avg)	< 2 seconds	13.1 ms	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	25.9 ms	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 20 req/s	358.9 req/s	Pass	Excellent request handling capacity
4	Error Rate	<1%	0.0%	Pass	No request failures
5	CPU Utilization	<80%	61%	Pass	Efficient CPU usage
6	Memory Utilization	<80%	57%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

• The HDI Predictor system successfully processed concurrent user requests.
• Average prediction response time remained below 20 milliseconds under load.
• No failed prediction requests were observed during testing (0% error rate).
• Flask application remained stable under moderate and concurrent workloads.
• Gradient Boosting Regressor model delivered fast and accurate predictions without performance degradation.

Bottlenecks Identified

• Minor increase in response time under concurrency spike (maximum latency reached 61.4 ms).
• Initial server startup/loading of sklearn models and scalers might introduce a minor delay on the very first request.

Optimization Steps Taken

• Scaled features efficiently using StandardScaler in memory.
• Pre-loaded the trained best machine learning model and scaler at startup in Flask app context (app.py), avoiding file I/O overhead on each request.
• Implemented robust HTML form inputs validation before feeding data to the model.
• Optimized NumPy operations to ensure fast prediction calculations.

Step 5: Screenshots / Evidence